

VxLEARN Networks

Networking & Cybersecurity Track
Simulated Employment Program

Lab Report: Using Windows PowerShell

Prepared by:
Kudzaishe Majeza
Junior Network Engineer – VxLEARN Networks

Mentor:
Titus Majeza
Senior Network Engineer

Date: 20 February 2026

Table of Contents

Introduction	3
Objectives	3
Part 1: Access PowerShell Console	3
Step 1: Open PowerShell and Command Prompt.....	3
Part 2: Explore Command Prompt and PowerShell Commands	4
Step 1: Run the dir Command.....	4
Observation:.....	4
Step 2: Test Other Commands	5
Results:.....	5
Part 3: Explore Cmdlets.....	7
Step 1: Identify Alias for dir	7
Output:.....	7
Conclusion:.....	7
Part 4: Explore netstat Command in PowerShell.....	8
Step 1: View netstat Options	8
Step 2: View Routing Table	9
Observation:.....	9
Step 3: Run Elevated PowerShell	10
Step 4: Match PID in Task Manager	11
Information Found:	11
Part 5: Empty Recycle Bin Using PowerShell	12
Step 1: Verify Files in Recycle Bin.....	12
Step 2: Clear Recycle Bin Using PowerShell.....	13
Result:	13
Reflection Question	14
How can PowerShell help a security analyst?.....	14
Conclusion.....	14

Introduction

In this lab, I explored Windows PowerShell and compared it to the traditional Command Prompt. The goal was to understand how PowerShell works, how cmdlets function, how network connections can be viewed using netstat, and how PowerShell can automate tasks such as clearing the Recycle Bin.

PowerShell is more powerful than Command Prompt because it is not only a command-line interface but also a scripting and automation tool that interacts directly with the Windows operating system.

Objectives

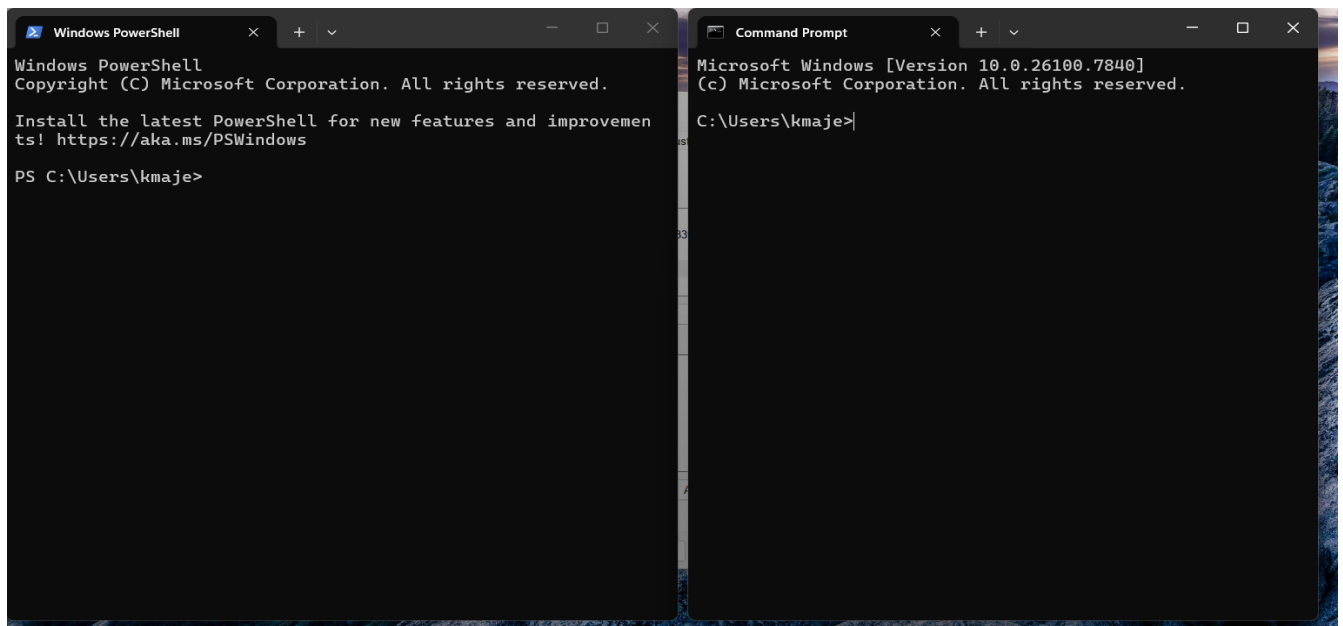
- Access PowerShell console
- Compare Command Prompt and PowerShell commands
- Explore PowerShell cmdlets
- Use netstat in PowerShell
- Empty the Recycle Bin using PowerShell

Part 1: Access PowerShell Console

Step 1: Open PowerShell and Command Prompt

I clicked **Start**, searched for **PowerShell**, and opened it. Then I searched for **Command Prompt** and opened it as well.

This allowed me to compare both interfaces side by side.



Part 2: Explore Command Prompt and PowerShell Commands

Step 1: Run the dir Command

I entered the following command in both windows:

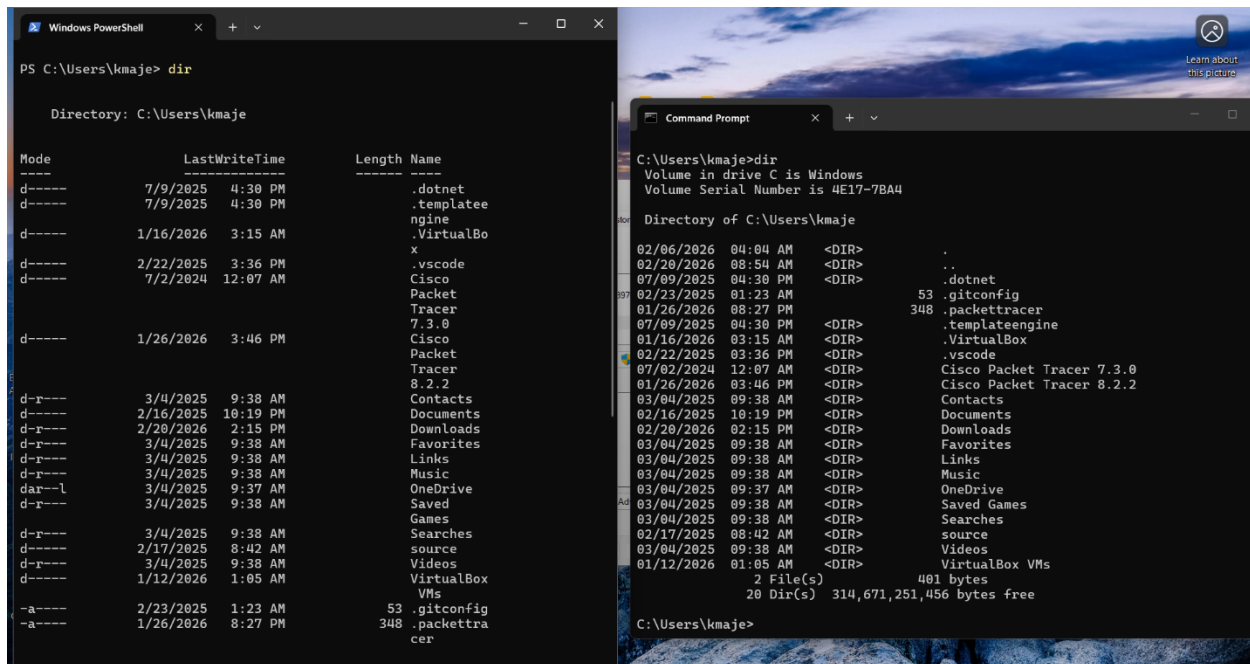
```
Dir
```

Observation:

Both PowerShell and Command Prompt displayed the list of files and folders in the current directory. However:

- Command Prompt displayed a basic text list.
- PowerShell displayed a more structured, object-based output.

This shows that PowerShell supports traditional commands but processes them differently.



Step 2: Test Other Commands

I tested:

ping

ipconfig

cd

Results:

- ping worked in both consoles.
- ipconfig displayed network configuration.
- cd allowed me to change directories.

PowerShell supports many Command Prompt commands for compatibility.

```
Windows PowerShell
PS C:\Users\kmaje> ping

Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
          [-r count] [-s count] [[-j host-list] | [-k host-list]]
          [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
          [-4] [-6] target_name

Options:
    -t                Ping the specified host until stopped.
                      To see statistics and continue - type Control-Break;
                      To stop - type Control-C.
    -a                Resolve addresses to hostnames.
    -n count          Number of echo requests to send.
    -l size           Send buffer size.
    -f               Set Don't Fragment flag in packet (IPv4-only).
    -i TTL            Time To Live.
    -v TOS            Type Of Service (IPv4-only. This setting has been depre
cated
                    and has no effect on the type of service field in the I
P
                    Header).
    -r count          Record route for count hops (IPv4-only).
    -s count          Timestamp for count hops (IPv4-only).
    -j host-list      Loose source route along host-list (IPv4-only).
    -k host-list      Strict source route along host-list (IPv4-only).
    -w timeout        Timeout in milliseconds to wait for each reply.
    -R               Use routing header to test reverse route also (IPv6-onl
y).
                    Per RFC 5095 the use of this routing header has been
                    deprecated. Some systems may drop echo requests if
```

```
Windows PowerShell
PS C:\Users\kmaje> ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::2878:839d:9d95:cbc%10
    IPv4 Address. . . . . : 192.168.56.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 

Wireless LAN adapter Local Area Connection* 1:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : 

Wireless LAN adapter WiFi:

    Connection-specific DNS Suffix  . : 
    IPv6 Address. . . . . : fdc2:1e24:c5ef:9100:e7f9:7030:8d00:
c1c6
    Temporary IPv6 Address. . . . . : fdc2:1e24:c5ef:9100:dd89:72a9:5a7e:
8daa
    Link-local IPv6 Address . . . . . : fe80::f89c:c03a:d11d:2d40%3
    IPv4 Address. . . . . : 192.168.8.70
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.8.1
PS C:\Users\kmaje> |
```

Part 3: Explore Cmdlets

PowerShell commands are called **cmdlets**, and they follow a:

Verb-Noun format

Example:

Get-ChildItem

Step 1: Identify Alias for dir

I entered:

```
Get-Alias dir
```

Output:

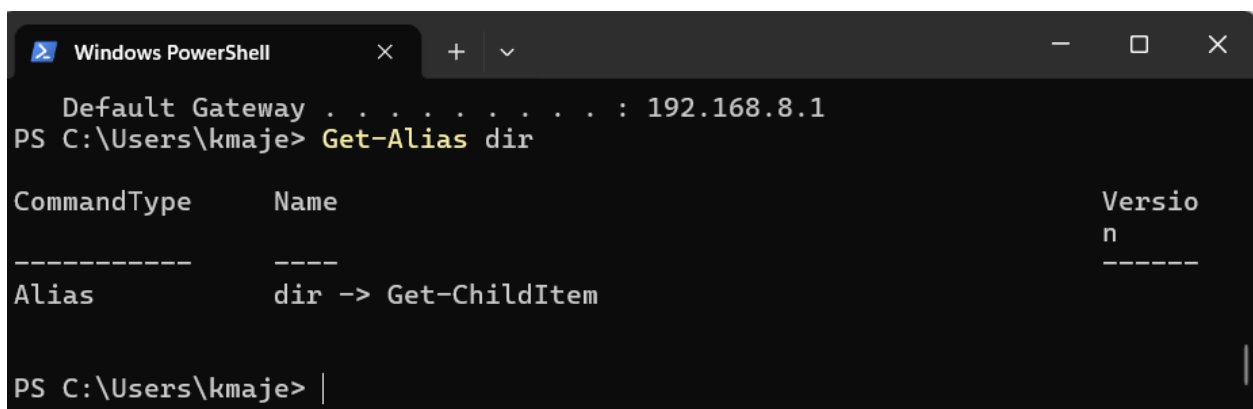
```
Alias dir -> Get-ChildItem
```

Conclusion:

The PowerShell equivalent of dir is:

```
Get-ChildItem
```

This shows that PowerShell uses object-based commands instead of simple text-based commands.



```
Windows PowerShell
Default Gateway . . . . . : 192.168.8.1
PS C:\Users\kmaje> Get-Alias dir

CommandType      Name
-----
Alias            dir -> Get-ChildItem

PS C:\Users\kmaje> |
```

Part 4: Explore netstat Command in PowerShell

Step 1: View netstat Options

I entered:

```
netstat -h
```

This displayed all available parameters such as:

- -a (all connections)
- -b (executable name)
- -n (numeric addresses)
- -o (process ID)

Step 2: View Routing Table

I entered:

```
netstat -r
```

Observation:

The IPv4 Gateway displayed was:

```
192.168.8.1
```

This is the default gateway used to access networks outside the local network.

```
Windows PowerShell
PS C:\Users\kmaje> netstat -r

Interface List
10...0a 00 27 00 00 0a .....VirtualBox Host-Only Ethernet Adapter
14...42 1a 58 b7 35 8b .....Microsoft Wi-Fi Direct Virtual Adapter
3...40 1a 58 b7 35 8b .....Realtek RTL8852BE-VS WiFi 6 802.11ax PCIe Adapter
1.....Software Loopback Interface 1

IPv4 Route Table

Active Routes:
Network Destination    Netmask          Gateway          Interface        Metric
0.0.0.0                0.0.0.0          192.168.8.1      192.168.8.70     50
127.0.0.0              255.0.0.0        On-link          127.0.0.1        331
127.0.0.1              255.255.255.255  On-link          127.0.0.1        331
127.255.255.255        255.255.255.255  On-link          127.0.0.1        331
192.168.8.0            255.255.255.0    On-link          192.168.8.70     306
192.168.8.70          255.255.255.255  On-link          192.168.8.70     306
192.168.8.255          255.255.255.255  On-link          192.168.8.70     306
192.168.56.0           255.255.255.0    On-link          192.168.56.1     281
192.168.56.1           255.255.255.255  On-link          192.168.56.1     281
192.168.56.255         255.255.255.255  On-link          192.168.56.1     281
224.0.0.0              240.0.0.0        On-link          127.0.0.1        331
224.0.0.0              240.0.0.0        On-link          192.168.56.1     281
224.0.0.0              240.0.0.0        On-link          192.168.8.70     306
255.255.255.255        255.255.255.255  On-link          127.0.0.1        331
255.255.255.255        255.255.255.255  On-link          192.168.56.1     281
255.255.255.255        255.255.255.255  On-link          192.168.8.70     306
```

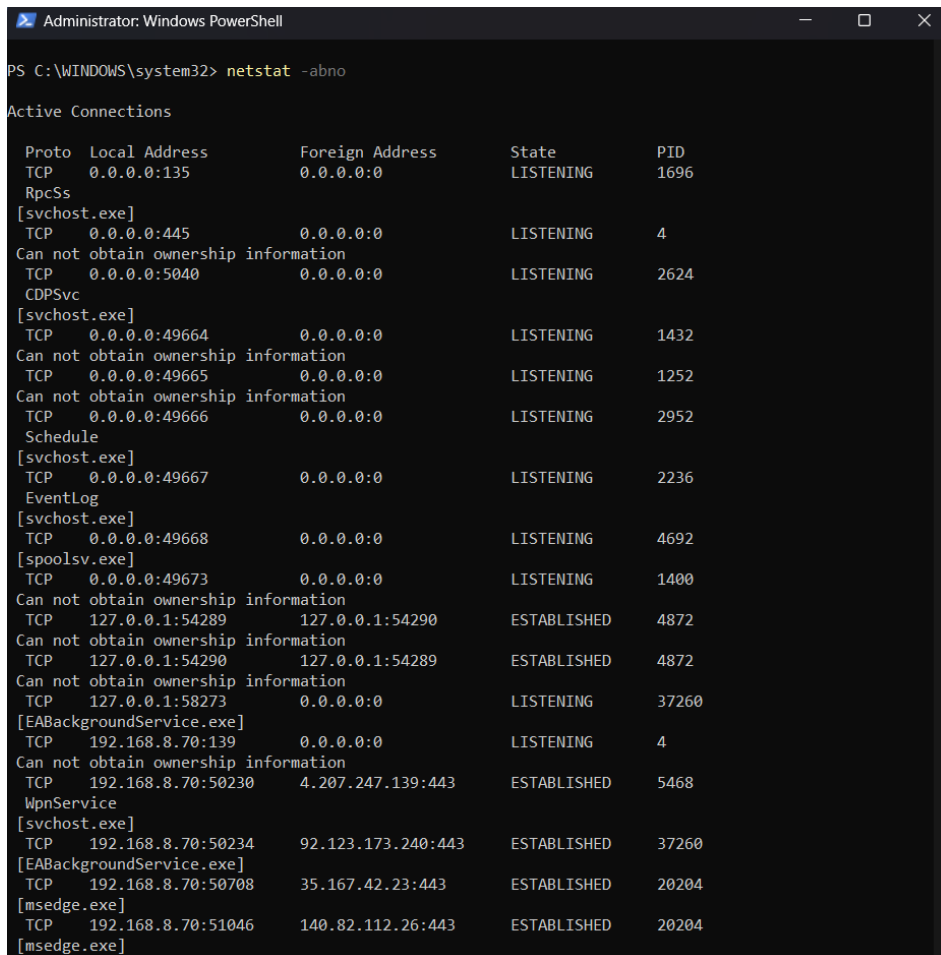
Step 3: Run Elevated PowerShell

I opened PowerShell as Administrator and entered:

```
netstat -abno
```

This command shows:

- Active connections
- Listening ports
- Executable names
- Process IDs (PIDs)



```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> netstat -abno

Active Connections

Proto Local Address           Foreign Address         State       PID
TCP   0.0.0.0:135              0.0.0.0:0               LISTENING   1696
RpcSs
[svchost.exe]
TCP   0.0.0.0:445              0.0.0.0:0               LISTENING   4
Can not obtain ownership information
TCP   0.0.0.0:5040             0.0.0.0:0               LISTENING   2624
CDPSvc
[svchost.exe]
TCP   0.0.0.0:49664            0.0.0.0:0               LISTENING   1432
Can not obtain ownership information
TCP   0.0.0.0:49665            0.0.0.0:0               LISTENING   1252
Can not obtain ownership information
TCP   0.0.0.0:49666            0.0.0.0:0               LISTENING   2952
Schedule
[svchost.exe]
TCP   0.0.0.0:49667            0.0.0.0:0               LISTENING   2236
EventLog
[svchost.exe]
TCP   0.0.0.0:49668            0.0.0.0:0               LISTENING   4692
[spoolsv.exe]
TCP   0.0.0.0:49673            0.0.0.0:0               LISTENING   1400
Can not obtain ownership information
TCP   127.0.0.1:54289          127.0.0.1:54290         ESTABLISHED 4872
Can not obtain ownership information
TCP   127.0.0.1:54290          127.0.0.1:54289         ESTABLISHED 4872
Can not obtain ownership information
TCP   127.0.0.1:58273          0.0.0.0:0               LISTENING   37260
[EABackgroundService.exe]
TCP   192.168.8.70:139         0.0.0.0:0               LISTENING   4
Can not obtain ownership information
TCP   192.168.8.70:50230       4.207.247.139:443       ESTABLISHED 5468
WpnService
[svchost.exe]
TCP   192.168.8.70:50234       92.123.173.240:443       ESTABLISHED 37260
[EABackgroundService.exe]
TCP   192.168.8.70:50708       35.167.42.23:443        ESTABLISHED 20204
[msedge.exe]
TCP   192.168.8.70:51046       140.82.112.26:443       ESTABLISHED 20204
[msedge.exe]
```

Step 4: Match PID in Task Manager

I opened Task Manager → Details tab and located a PID shown in netstat.

Example:

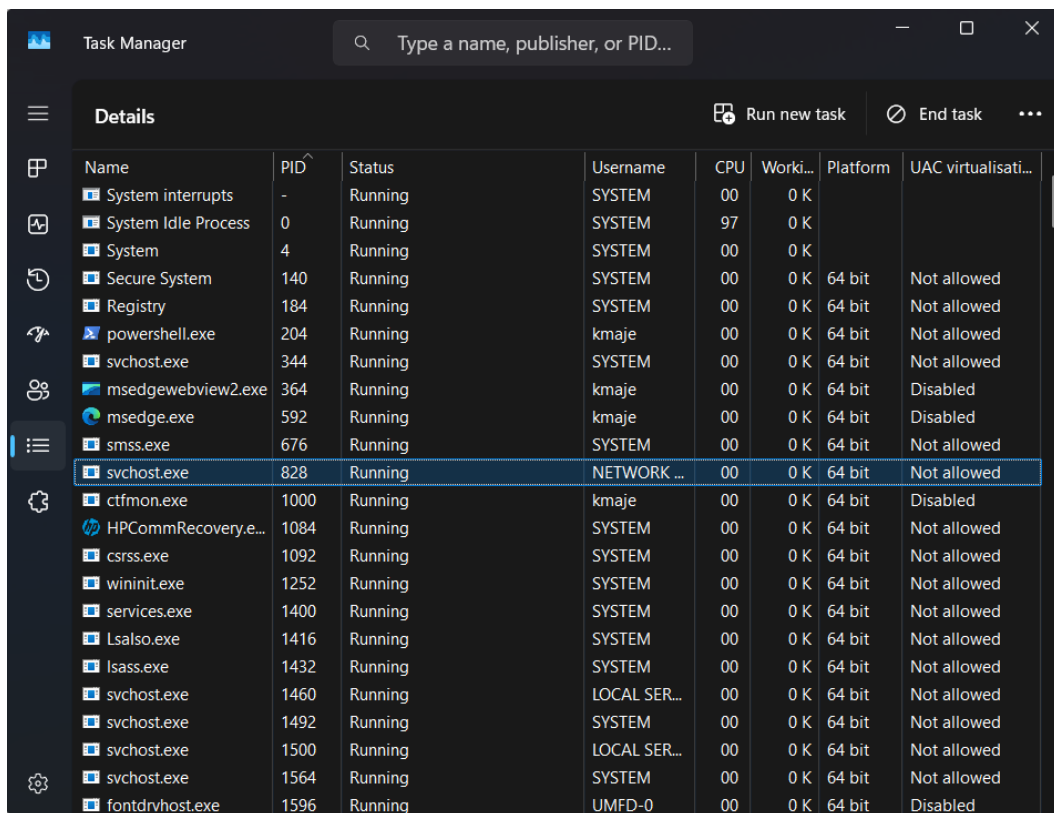
PID 756 → svchost.exe

Information Found:

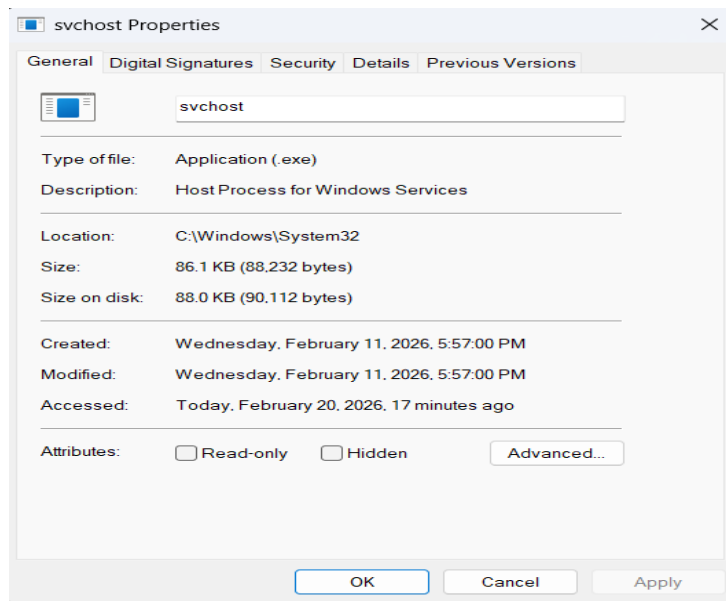
From Task Manager and Properties I could see:

- Process name
- CPU usage
- Memory usage
- Executable location
- Description of the service

This helps security analysts identify suspicious processes.



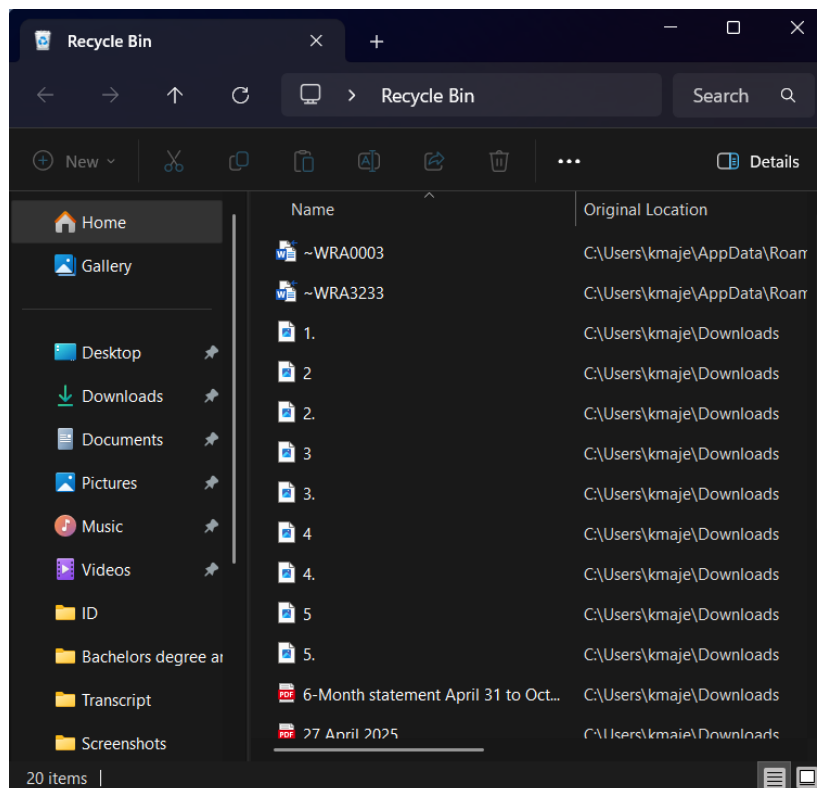
Name	PID	Status	Username	CPU	Worki...	Platform	UAC virtualisati...
System interrupts	-	Running	SYSTEM	00	0 K		
System Idle Process	0	Running	SYSTEM	97	0 K		
System	4	Running	SYSTEM	00	0 K		
Secure System	140	Running	SYSTEM	00	0 K	64 bit	Not allowed
Registry	184	Running	SYSTEM	00	0 K	64 bit	Not allowed
powershell.exe	204	Running	kmaje	00	0 K	64 bit	Not allowed
svchost.exe	344	Running	SYSTEM	00	0 K	64 bit	Not allowed
msedgewebview2.exe	364	Running	kmaje	00	0 K	64 bit	Disabled
msedge.exe	592	Running	kmaje	00	0 K	64 bit	Disabled
smss.exe	676	Running	SYSTEM	00	0 K	64 bit	Not allowed
svchost.exe	828	Running	NETWORK ...	00	0 K	64 bit	Not allowed
ctfmon.exe	1000	Running	kmaje	00	0 K	64 bit	Disabled
HPCommRecovery.e...	1084	Running	SYSTEM	00	0 K	64 bit	Not allowed
csrss.exe	1092	Running	SYSTEM	00	0 K	64 bit	Not allowed
wininit.exe	1252	Running	SYSTEM	00	0 K	64 bit	Not allowed
services.exe	1400	Running	SYSTEM	00	0 K	64 bit	Not allowed
Lsalso.exe	1416	Running	SYSTEM	00	0 K	64 bit	Not allowed
lsass.exe	1432	Running	SYSTEM	00	0 K	64 bit	Not allowed
svchost.exe	1460	Running	LOCAL SER...	00	0 K	64 bit	Not allowed
svchost.exe	1492	Running	SYSTEM	00	0 K	64 bit	Not allowed
svchost.exe	1500	Running	LOCAL SER...	00	0 K	64 bit	Not allowed
svchost.exe	1564	Running	SYSTEM	00	0 K	64 bit	Not allowed
fontdrvhost.exe	1596	Running	UMFD-0	00	0 K	64 bit	Disabled



Part 5: Empty Recycle Bin Using PowerShell

Step 1: Verify Files in Recycle Bin

I opened the Recycle Bin and confirmed there were files inside.



Step 2: Clear Recycle Bin Using PowerShell

I entered:

clear-recyclebin

PowerShell asked for confirmation:

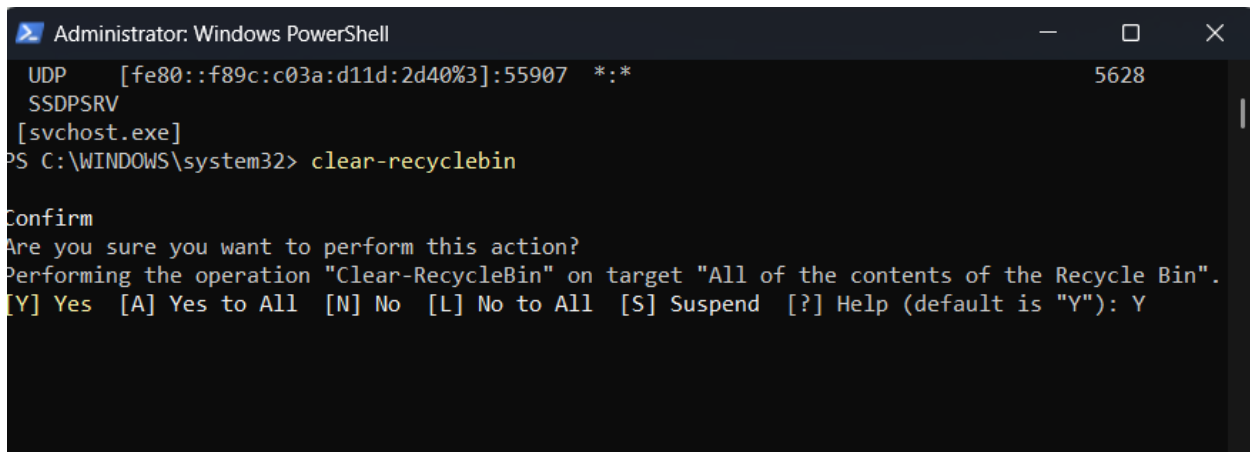
[Y] Yes

I typed:

Y

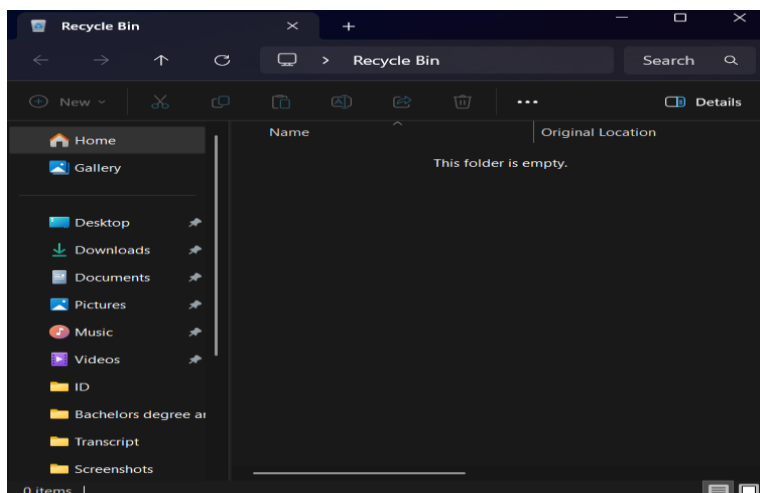
Result:

All files were permanently deleted from the Recycle Bin.



```
Administrator: Windows PowerShell
UDP [fe80::f89c:c03a:d11d:2d40%3]:55907 *:.* 5628
SSDPSRV
[svchost.exe]
PS C:\WINDOWS\system32> clear-recyclebin

Confirm
Are you sure you want to perform this action?
Performing the operation "Clear-RecycleBin" on target "All of the contents of the Recycle Bin".
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "Y"): Y
```



Reflection Question

How can PowerShell help a security analyst?

PowerShell can automate tasks such as:

- Checking active network connections:

`Get-NetTCPConnection`

- Listing running processes:

`Get-Process`

- Viewing system logs:

`Get-EventLog`

- Checking user accounts:

`Get-LocalUser`

PowerShell makes it easier to monitor systems, detect suspicious activity, automate security checks, and manage multiple machines efficiently.

Conclusion

In this lab, I learned that PowerShell is much more powerful than Command Prompt. It supports traditional commands but also introduces cmdlets that work with objects rather than plain text.

I explored:

- File system commands
- Network statistics
- Running processes
- Routing tables
- Automation commands

PowerShell is an essential tool for system administrators and security analysts because it allows automation, monitoring, and deep system control from the command line.